

004.056.55

*И. В. Кудрявцев¹, В. В. Горгадзе¹, А. В. Беленов¹*¹Московский физико-технический институт (национальный исследовательский университет)

Безопасное хранение и восстановление пользовательских данных с помощью мультиагентной схемы в Cosmos

Решение проблемы уязвимости пользователей к потере приватных ключей для безопасного хранения и восстановления данных в блокчейн-системах является критически важным, так как потеря ключа означает безвозвратную утрату доступа к данным и цифровым активам. Предложен метод восстановления зашифрованных данных через группу доверенных лиц, основанный на математическом алгоритме разделения секрета Шамира. Ключ делится на части и распределяется между участниками так, что для восстановления требуется согласие минимального числа из них. Система реализована для блокчейна Cosmos и обеспечивает защиту от компрометации отдельных участников при сохранении возможности восстановления данных. Решение не требует доверия к централизованным сервисам и подтверждено экспериментально на работающем прототипе.

: блокчейн, Cosmos, схема Шамира, абстракция аккаунта, восстановление ключей, мультиагентная система

*I. V. Kudryavtsev¹, V. V. Gorgadze¹, A. V. Belenov¹*¹Moscow Institute of Physics and Technology (National Research University)

Secure Storage and Recovery of User Data Using Multi-Agent Scheme in Cosmos

Solving the problem of user vulnerability to private key loss for secure storage and data recovery in blockchain systems is critically important, as key loss means irreversible loss of access to data and digital assets. A method for recovering encrypted data through a group of trusted parties is proposed, based on Shamir's Secret Sharing mathematical algorithm. The key is divided into shares and distributed among participants so that recovery requires agreement of a minimum number of them. The system is implemented for Cosmos blockchain and provides protection against compromise of individual participants while maintaining data recovery capability. The solution requires no trust in centralized services and is experimentally validated on a working prototype.

Key words: blockchain, Cosmos, Shamir's scheme, account abstraction, key recovery, multi-agent system

1. Введение

Развитие децентрализованных технологий и блокчейн-сетей открыло новые горизонты в области цифровой идентичности и управления пользовательскими данными. Однако вместе с этим появились и новые вызовы, связанные с безопасностью и сохранностью доступа к данным. Одной из ключевых проблем остаётся высокая уязвимость пользователей к потере приватных ключей, которая в традиционных блокчейн-системах означает полную и безвозвратную утрату доступа к аккаунту и связанным с ним цифровым активам и конфиденциальным данным.

Современные подходы к решению этой проблемы включают механизмы:

© Кудрявцев И. В., Горгадзе В. В., Беленов А. В., 2025

© Федеральное государственное автономное образовательное учреждение высшего образования «Московский физико-технический институт (национальный исследовательский университет)», 2025

- 1) социального восстановления (social recovery) — передачу части полномочий по восстановлению доверенной группе лиц;
- 2) абстракции аккаунта (account abstraction) — реализацию аутентификации и авторизации через программируемую логику смарт-контрактов.

Несмотря на перспективность этих решений, существующие реализации зачастую ограничиваются восстановлением лишь доступа к управлению аккаунтом, но не охватывают проблему восстановления зашифрованных пользовательских данных. Кроме того, они требуют предварительной конфигурации и не всегда обеспечивают должный уровень безопасности в сценариях с частичной компрометацией доверенных участников.

Цель данной работы — разработка и реализация расширенного подхода к безопасному хранению и восстановлению пользовательских данных в экосистеме **Cosmos**. Предлагаемое решение сочетает в себе мультиагентную схему на основе социальной модели доверия с использованием алгоритма разделения секрета Шамира (**Shamir Secret Sharing**) и хранением зашифрованного ключа непосредственно в состоянии смарт-контракта. Такая архитектура позволяет пользователю не только гибко управлять доступом, но и восстанавливать утраченные зашифрованные данные даже при потере собственного ключа, опираясь на сеть доверенных лиц.

1.1. Научный и практический вклад

Основной вклад работы:

- 1) Формализована пороговая модель восстановления зашифрованных пользовательских данных (в отличие от существующих подходов, ограничивающихся восстановлением доступа к аккаунту и активам) на основе интеграции схемы Шамира и механизма социального восстановления в архитектуру абстрактного аккаунта экосистемы **Cosmos**. Модель обеспечивает полную децентрализацию через сеть независимых guardians без зависимости от Web2-провайдеров.
- 2) Проведён количественный анализ безопасности и отказоустойчивости: доказана конфиденциальность при компрометации до $k - 1$ участников и функциональность при отказе до $n - k$ участников для конфигурации (k, n) -пороговой схемы.
- 3) Реализован и экспериментально подтверждён работоспособный прототип на базе **CosmWasm** с открытым исходным кодом на локальной тестовой сети **Cosmos SDK**, демонстрирующий практическую применимость предложенного подхода.

Принципиальное отличие от аналогов: Argent Wallet [1] восстанавливает доступ к аккаунту и активам, Leap/Keplr используют централизованные OAuth-провайдеры, но ни одно из этих решений не восстанавливает зашифрованные пользовательские данные. Предложенная система обеспечивает восстановление зашифрованных данных через децентрализованную сеть guardians без зависимости от внешних сервисов.

2. Связанные работы и технологии

Вопрос обеспечения безопасности доступа и восстановления пользовательских данных активно исследуется в контексте блокчейн-технологий. Наиболее заметные направления включают абстракцию аккаунтов, социальное восстановление и методы распределённого хранения секретов.

2.1. Абстракция аккаунта

Идея абстракции аккаунта (account abstraction) была впервые предложена в экосистеме **Ethereum**. Основная цель — перенести логику аутентификации и управления средствами с уровня протокола на уровень смарт-контрактов. Это позволяет реализовывать произвольные схемы доступа, включая мультиподпись, лимиты, кастомные правила и, в частности, механизмы восстановления.

Развитие концепции абстракции аккаунта в **Ethereum** прошло несколько этапов. Изначально предложенный Виталиком Бутериным в 2016 году EIP-86 [2] пытался ввести кошельки смарт-контрактов в виде «контрактов пересылки», которые принимали бы транзакции только с адреса «точки входа». Однако данное предложение требовало значительных изменений в протоколе **Ethereum** и в итоге не было реализовано. Частично идеи EIP-86 были воплощены в EIP-1014 [3], который ввел опкод CREATE2 в 2018 году.

Дальнейшие попытки реализации абстракции аккаунта включали EIP-2938 [4] (2020 год) и EIP-3074, которые также требовали добавления новых опкодов в EVM и изменения основного протокола. Необходимость в стандарте, не влияющем на сам протокол, привела к созданию ERC-4337 [5].

Наиболее зрелой реализацией этого подхода стал стандарт **ERC-4337**, позволяющий использовать контрактные аккаунты без модификации консенсусного уровня. Он вводит сущности вроде **EntryPoint**, **Bundler**, **Paymaster** и других, позволяющих эмулировать транзакции пользователей через контракт. Однако такие схемы увеличивают сложность и могут приводить к централизации, особенно в части доверия к **Bundler** и **Paymaster**.

В экосистеме **Cosmos** подобная функциональность реализуется с помощью смарт-контрактов на базе **CosmWasm** [6] и специализированных модулей SDK. В рамках предыдущих работ автором был разработан смарт-контракт, реализующий модель абстрактного аккаунта с поддержкой социальной модели восстановления (social recovery), в которой группа доверенных лиц (guardians) может восстановить доступ пользователя при достижении порога голосов.

2.2. Социальное восстановление (Social Recovery)

Социальное восстановление представляет собой механизм смены приватного ключа для владельца средств в случае его потери или компрометации [7, 8]. При регистрации абстрактного аккаунта владелец определяет список доверенных опекунов (guardians) и пороговое количество их подписей, необходимое для инициации процесса восстановления. Концептуальное обоснование необходимости широкого adoption social recovery-моделей представлено в работе Бутерина [9], где подчёркивается, что данный подход обеспечивает баланс между удобством использования и децентрализацией, позволяя пользователям сохранять контроль при минимизации единой точки отказа.

Основные риски включают социальную инженерию, потерю ключей опекунами, сложность мониторинга участников и возможность заговорческих действий. Несмотря на это, механизм социального восстановления значительно повышает устойчивость системы к потере доступа.

Наиболее известным практическим примером является реализация **Argent Wallet** в **Ethereum**, использующая схожую модель на основе контрактных аккаунтов и multisig-механизма. Стандарт **ERC-7093** [7] формализует интерфейс для социального восстановления в экосистеме **Ethereum**. В **Cosmos** подобные подходы пока остаются экспериментальными и не получили широкого распространения, что делает тему особенно актуальной.

2.3. Разделение секрета: Shamir Secret Sharing

Классический алгоритм **Shamir Secret Sharing (SSS)**, предложенный Ади Шамиром в 1979 году [10], позволяет надёжно распределить секрет между n участниками таким образом, что только при наличии минимум t частей (порог t) секрет может быть восстановлен.

Этот механизм широко применяется в криптографии, включая распределённые хранилища и системы резервного копирования.

Математическая основа SSS базируется на интерполяции полиномов: секрет кодируется как свободный член полинома степени $t - 1$ над конечным полем. Каждая доля представляет собой точку на этом полиноме. Знание любых t точек позволяет восстановить исходный полином и, следовательно, секрет. Важным свойством схемы является то, что знание менее t точек не даёт никакой информации о секрете — это обеспечивает теоретико-информационную безопасность.

Практические применения SSS в контексте децентрализованных систем охватывают различные области: от резервного копирования криптографических ключей до распределённого хранения конфиденциальных данных и блокчейн-приложений. Детальный анализ существующих решений на основе SSS представлен в следующем разделе.

2.4. Анализ существующих решений

Для позиционирования предлагаемого подхода проведён сравнительный анализ существующих решений в области восстановления доступа к данным в блокчейн-системах.

Коммерческие кошельки. **Argent Wallet** [1] представляет наиболее зрелую реализацию социального восстановления в экосистеме Ethereum. Система использует смарт-контракты для управления аккаунтом и позволяет группе guardians голосовать за смену владельца при достижении порогового значения. Восстанавливается доступ к аккаунту и находящимся на нём активам, однако зашифрованные пользовательские данные, защищённые исходным ключом шифрования, остаются недоступными.

Leap Wallet и **Keplr** — ведущие кошельки экосистемы Cosmos — предоставляют механизмы социальной аутентификации через Web2-провайдеры (Google, GitHub). Такой подход упрощает onboarding, но создаёт зависимость от централизованных сервисов и не использует пороговую модель распределения секрета. При компрометации OAuth-провайдера или его недоступности восстановление становится невозможным.

Исследования и практические решения на основе SSS. **Vault12** [11] представляет платформу децентрализованного хранения криптоактивов, использующую иерархическую схему Шамира и mesh-сеть доверенных устройств. Подход схож с предлагаемым в использовании социальной модели доверия, однако фокусируется на восстановлении доступа к активам, а не к зашифрованным пользовательским данным, и не интегрирован с блокчейн-контрактами.

Grid+ [12] предложил восстановление ключей через множественных внешних агентов (собственное устройство, мобильное устройство пользователя, внешний сервер). Решение ограничено зависимостью от специализированного оборудования и централизованных сервисов, что противоречит принципам децентрализации.

Существенный прогресс в развитии схем social recovery продемонстрирован в работе **Pedin et al.** [15], предложивших smart contract-based протокол восстановления активов, сочетающий SSS, Merkle-структуры для верификации долей и биометрическое шифрование для защиты от потенциального сговора guardians. Ключевое преимущество — guardians не совершают on-chain транзакции, что снижает затраты газа (в 2–3 раза по сравнению с Argent при росте числа guardians) и повышает приватность. Экспериментальная оценка на Ethereum показывает константные затраты восстановления (~56k газа) независимо от числа guardians. Однако схема ориентирована на восстановление цифровых активов, а не пользовательских данных.

Sharvot [13] демонстрирует применение SSS для блокчейн-голосования, обеспечивая конфиденциальность голосов до момента подведения итогов. Хотя работа подтверждает применимость SSS в блокчейн-контексте, область применения (голосование) значительно отличается от восстановления данных.

Singh et al. [14] представляет подход, близкий по постановке задачи: восстановление приватных ключей через SSS в контексте Self-Sovereign Identity на Ethereum. Механизм основан на когнитивных данных (персональные вопросы), а не на мультиагентной модели. Восстанавливается ключ, но не пользовательские данные.

Сравнительная таблица. В табл. 1 представлено сравнение существующих решений по ключевым параметрам. Важно различать объект восстановления:

- **Аккаунт/Активы** — восстановление доступа к блокчейн-аккаунту и находящимся на нём активам путём смены владельца или генерации нового ключа управления. Пользователь получает контроль над токенами и возможность совершать транзакции, однако зашифрованные пользовательские данные, защищённые исходным ключом шифрования, остаются недоступными.
- **Ключ** — восстановление самого приватного ключа (обычно для блокчейн-аутентификации), но без механизма восстановления зашифрованных данных.
- **Данные** — восстановление ключа шифрования для доступа к зашифрованным пользовательским данным (файлам, документам, конфиденциальной информации), хранящимся в смарт-контракте. При этом также восстанавливается доступ к аккаунту и активам.

Т а б л и ц а 1

Сравнительный анализ решений для восстановления в блокчейн-системах

Решение	Модель	Восстанавливается	Web2	Cosmos
Argent Wallet	Social recovery	Аккаунт	Нет	Нет
Pedin [15]	SSS+Merkle+Bio	Активы	Нет	Нет
Leap/Keplr	OAuth	Аккаунт	Да	Да
Vault12 [11]	Mesh network	Активы	Нет	Нет
Grid+ [12]	External agents	Ключ	Частично	Нет
Sharvot [13]	SSS+Voting	Голоса	Нет	Нет
Singh [14]	Questions+SSS	Ключ	Нет	Нет
Данная работа	Guardian+SSS	Данные	Нет	Да

Как видно из таблицы, существующие решения восстанавливают доступ к блокчейн-аккаунту и активам (Argent, Pedin, Leap/Keplr, Vault12) или сам приватный ключ (Grid+, Singh), но не решают задачу восстановления ключа шифрования для доступа к зашифрованным пользовательским данным. Предлагаемое решение впервые реализует комплексную модель восстановления: не только доступ к аккаунту и активам, но и к зашифрованным пользовательским данным в контексте экосистемы Cosmos, с полной децентрализацией и без зависимости от Web2-сервисов.

Основная проблема, которую решает данная работа: необходимо обеспечить безопасное хранение и возможность восстановления пользовательских данных в децентрализованной сети при условии, что пользователь может потерять доступ к приватному ключу, но должен иметь возможность восстановить доступ к зашифрованным данным через доверенных агентов.

Проведённый анализ показывает, что существующие решения не полностью решают данную проблему в контексте экосистемы Cosmos, что обосновывает необходимость разработки специализированного подхода.

3. Архитектура предлагаемого решения

3.1. Общая схема

Предлагаемое решение основано на расширении концепции абстрактного аккаунта в экосистеме Cosmos с интеграцией схемы разделения секрета Шамира. Архитектура включает следующие ключевые компоненты: смарт-контракт для управления аккаунтом, систему доверенных агентов (guardians) и механизм восстановления через пороговые подписи.

Основные принципы работы системы базируются на следующих положениях:

- секрет разделяется на n частей с порогом восстановления k , где каждый guardian контролирует не более одной части;
- восстановление секрета возможно только при согласии минимум k guardians;
- компрометация менее k guardians не приводит к раскрытию секрета благодаря математическим свойствам схемы Шамира.

Безопасность системы основана на том, что секрет представляется как полином степени $k-1$, для восстановления которого необходимо знание минимум k точек. Знание менее k точек не позволяет однозначно определить полином и, следовательно, восстановить исходный секрет.

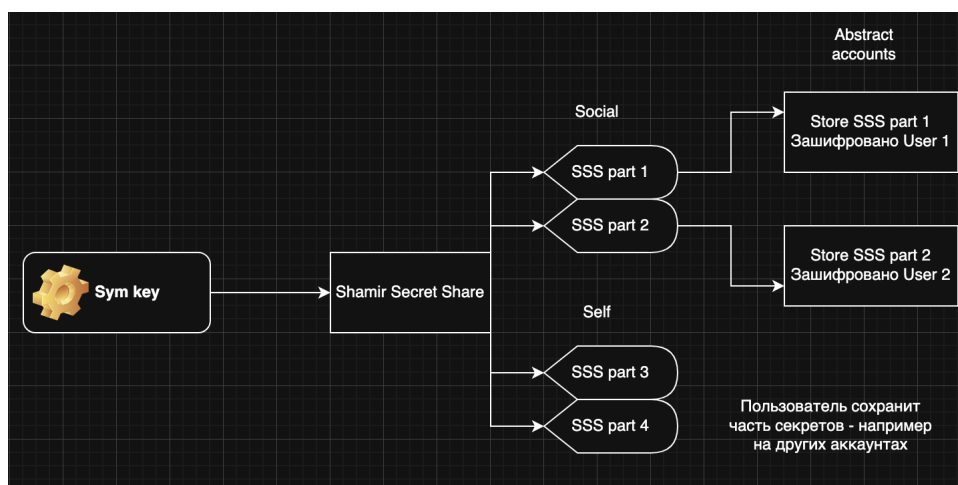


Рис. 1. Архитектура системы распределения симметричного ключа с использованием алгоритма Шамира

На рис. 1 представлена общая архитектура предлагаемой системы. Симметричный ключ (Sym key) обрабатывается алгоритмом Shamir Secret Sharing, который создает несколько частей секрета (SSS part 1, 2, 3, 4). Эти части распределяются между различными участниками системы: часть передается социальным guardians (которые хранят зашифрованные части в абстрактных аккаунтах), а часть может сохраняться самим пользователем на других устройствах или аккаунтах. Такое распределение обеспечивает отказоустойчивость системы при сохранении конфиденциальности исходного ключа.

Архитектура обеспечивает децентрализованное хранение частей секрета при сохранении возможности восстановления. Предлагаемая схема обеспечивает устойчивость к отказу до $n - k$ guardians и компрометации до $k - 1$ guardians одновременно.

3.2. Компоненты системы

Система состоит из следующих основных компонентов:

- 1) смарт-контракт на базе CosmWasm, хранящий зашифрованные данные пользователя и метаданные восстановления;

- 2) публичный ключ владельца аккаунта для верификации операций;
- 3) список доверенных агентов (guardians) с определенным пороговым количеством для восстановления;
- 4) механизм сборки секрета через агрегированные подписи guardians при инициации процедуры восстановления.

4. Детали реализации

Реализация основана на расширении смарт-контракта абстрактного аккаунта с интеграцией компонентов для работы со схемой Шамира и управления guardians. Исходный код опубликован в открытом репозитории на GitHub.

4.1. Структура данных смарт-контракта

Смарт-контракт реализован на языке Rust с использованием фреймворка CosmWasm. Состояние контракта включает следующие ключевые элементы хранения:

- **GUARDIANS**: список адресов доверенных агентов (`Vec<Addr>`);
- **THRESHOLD**: пороговое количество голосов для восстановления (`u64`);
- **VOTES**: карта голосов guardians с их подписями (`Map<&Addr, Binary>`);
- **COUNTS**: счетчик голосов по каждому новому ключу (`Map<&str, u64>`);
- **KEY_VALUE_STORE**: общее хранилище данных пользователя (`Map<&str, Binary>`);
- **DATA_SECRET**: зашифрованный симметричный ключ (`Binary`);
- **SHARES**: части секрета по схеме Шамира для каждого guardian (`Map<&Addr, Binary>`);
- **RECOVER_DATA**: временные данные для процесса восстановления (`Map<&Addr, Binary>`).

Смарт-контракт управляет метаданными и координирует голосование, а криптографические операции выполняются на стороне клиента.

4.2. Механизм разделения и хранения секрета

Процесс разделения и распределения секрета включает следующие этапы:

- 1) Исходный симметричный ключ разделяется на n частей с помощью алгоритма Шамира с пороговым значением k .
- 2) Каждая часть секрета шифруется публичным ключом соответствующего guardian для обеспечения конфиденциальности.
- 3) Зашифрованные части сохраняются в абстрактных аккаунтах каждого guardian в поле **SHARES**.
- 4) Каждая сохраненная часть подписывается публичным ключом guardian для обеспечения аутентичности и целостности данных.

4.3. Алгоритм восстановления секрета

Процесс восстановления реализован через систему голосования guardians и включает следующие этапы:

- 1) Пользователь инициирует запрос на восстановление, вызывая функцию `Recover{new_pubkey}` через любого из guardians.
- 2) Guardian вызывает функцию `recover()`, которая проверяет его статус, отсутствие повторного голосования и увеличивает счетчик голосов.
- 3) При достижении порогового количества голосов (`count >= threshold`) происходит автоматическое обновление публичного ключа аккаунта и очистка временных данных голосования.
- 4) Guardians могут отозвать свой голос с помощью функции `Revoke{}`, что уменьшает счетчик и удаляет их голос из системы.
- 5) Guardians размещают свои части секрета через функции `StoreShare{}` и `StoreRecoverData{}`.
- 6) Когда в `RECOVER_DATA` накапливается достаточное количество частей, клиентское приложение собирает их и восстанавливает исходный ключ шифрования.

Система предотвращает повторное голосование и обеспечивает атомарность через механизм подсчета голосов.

4.4. Особенности реализации

Клиентская часть. Пользовательский интерфейс реализован на React с использованием Chakra UI и Cosmos Kit. Основной интерфейс (рис. 2) включает управление балансом, guardians и данными.

Ключевые компоненты включают `CreateSecretButton` для генерации и распределения секрета по схеме Шамира, `WalletSection` для взаимодействия с абстрактным аккаунтом и API-эндпоинт `/api/generate-shares` для серверной генерации частей секрета. Процесс создания секрета (рис. 3) позволяет пользователю задать пороговое значение, ввести mnemonic-фразу и настроить параметры для каждого guardian.

Процесс создания секрета включает:

- 1) Пользователь указывает количество частей (`numShares`) и пороговое значение (`threshold`).
- 2) Система генерирует криптографический ключ и разделяет его на части с использованием схемы Шамира.
- 3) Каждая часть кодируется в base64 и может быть зашифрована асимметричным ключом соответствующего guardian.
- 4) Части сохраняются в смарт-контракте через функцию `StoreShare`.

Криптографические операции. Интеграция схемы Шамира выполняется на стороне клиента для обеспечения конфиденциальности. Используется библиотека `@cosmjs/crypto` с алгоритмом `Secp256k1` для:

- генерации ключевых пар из mnemonic-фраз;
- асимметричного шифрования частей секрета для каждого guardian;
- подписи транзакций и верификации подлинности операций.

Смарт-контракт хранит только зашифрованные данные и координирует восстановление, не имея доступа к расшифрованной информации.

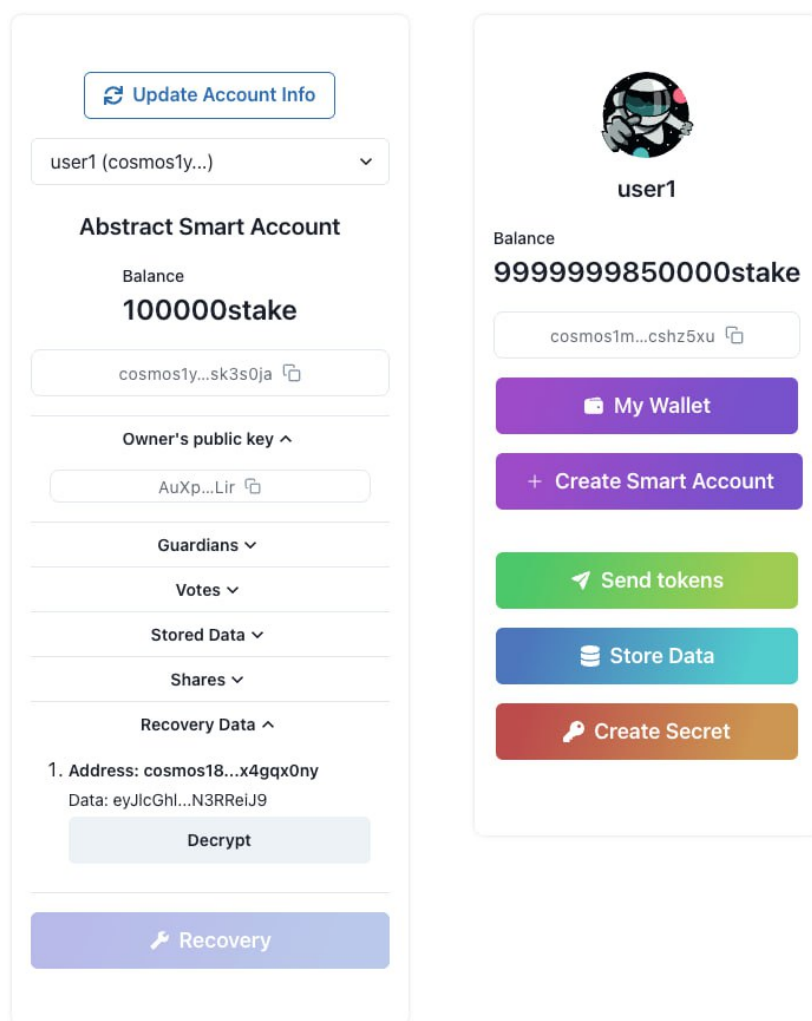


Рис. 2. Интерфейс абстрактного аккаунта с функциями управления guardians, голосования и хранения данных

5. Анализ безопасности и ограничений

5.1. Формальные свойства безопасности

Безопасность предлагаемой системы обеспечивается математическими свойствами схемы Шамира и архитектурными решениями:

- **Совершенная секретность:** Для схемы (k, n) -Шамира любая коалиция из менее чем k участников не получает никакой информации о секрете. Формально, для любых $k-1$ частей $\{s_{i_1}, s_{i_2}, \dots, s_{i_{k-1}}\}$ и любого возможного секрета S' , существует единственный полином степени $k-1$, проходящий через эти точки и имеющий S' в качестве свободного члена.
- **Отказоустойчивость:** Система остается функциональной при недоступности до $n-k$ guardians. При конфигурации $(3, 5)$ система выдерживает отказ любых двух участников.
- **Верифицируемость:** Каждая часть секрета подписана публичным ключом соответствующего guardian, что позволяет криптографически верифицировать подлинность данных через проверку подписи.

Create Secret

×

Threshold

2

Generated Key:

34b3afeb4474935d76ba819deb499397

Mnemonic

Enter your mnemonic phrase

Set Secret

Shares:

Share 1 ^

Recipient Address

Enter recipient address

Recipient Public Key

Enter recipient public key

Encoded Share:

eyJpbmRleCI6MSwieDEiOiIxIiwieTEiOiIyOTc5Nzg3ODQxNzYwND
U2MDI3MzgyNjI3NDM5NDkwMDY5MTE3NCIsIngyIjoiMSIsInkyIjoi
ODU2MzE4Mzg4NDU5NjkyMDI5OTQzMMD1NDIxMTc2NDM4NTYyMTgifQ
==

Raw Values:

x1: 1
y1: 29797878417604560273826274394900691174
x2: 1
v2: 85631838845969202994303542117643856218

Generate

Close

Рис. 3. Интерфейс создания и распределения секрета с использованием схемы Шамира

5.2. Количественная оценка рисков

Вероятностный анализ компрометации системы:

- **Вероятность компрометации:** При независимой вероятности компрометации каждого guardian равной p , вероятность компрометации всей системы составляет $P_{comp} = \sum_{i=k}^n \binom{n}{i} p^i (1-p)^{n-i}$. Для схемы (3, 5) и $p = 0.1$: $P_{comp} = 0.00856$ (0.856%).
- **Доступность системы:** Вероятность недоступности при независимой вероятности отказа guardian равной q составляет $P_{unavail} = \sum_{i=n-k+1}^n \binom{n}{i} q^i (1-q)^{n-i}$. Для $q = 0.2$: $P_{unavail} = 0.05792$ (5.792%).

5.3. Архитектурные ограничения

Выявлены следующие системные ограничения:

- **Социальная инженерия:** Атакующий может попытаться скоординированно воздействовать на k guardians. Противодействие: географическое и социальное разнесение участников, использование институциональных guardians.
- **Доступность данных:** Система требует онлайн-доступности минимум k guardians для восстановления. Решение: избыточность ($n > k$) и мониторинг доступности участников.
- **Временные окна атак:** Период между инициацией восстановления и его завершением создает окно уязвимости. Защита: временные ограничения на операции восстановления и механизмы отмены.

6. Экспериментальная оценка

Предложенное решение реализовано в виде прототипа на локальной тестовой сети Cosmos SDK. Работоспособность системы подтверждена экспериментально; исходный код смарт-контрактов CosmWasm и клиентского приложения опубликован в открытом доступе.

6.1. Архитектурные преимущества

Сравнительный анализ архитектурных особенностей предложенного решения с существующими подходами представлен в табл. 2.

Т а б л и ц а 2

Сравнительный анализ архитектурных характеристик

Характеристика	Данная работа	Argent	Pedin [15]
Восстанавливается	Данные + Аккаунт	Аккаунт	Аккаунт
Число транзакций	$k + 1$	k	Константа
Guardians on-chain	Да (голосование)	Да	Нет
Верификация долей	Подписи	Голосование	Merkle Tree
Вычислительная сложность	$O(k^2)$	$O(k)$	$O(k^2)$
Платформа	Cosmos	Ethereum	Ethereum

Ключевые особенности предложенной архитектуры:

- **Восстановление данных:** в отличие от Argent (аккаунт) и Pedin (активы), система восстанавливает зашифрованные пользовательские данные, что расширяет область применения за пределы управления средствами.
- **Децентрализация:** полная независимость от централизованных сервисов и Web2-провайдеров; guardians представляют собой независимые аккаунты в блокчейне Cosmos.
- **Вычислительная эффективность:** сложность восстановления секрета через интерполяцию Лагранжа составляет $O(k^2)$, где k — пороговое значение; для практических конфигураций ($k \leq 10$) издержки приемлемы.
- **Масштабируемость:** архитектура поддерживает динамическое изменение набора guardians без перераспределения всех долей секрета; добавление нового guardian требует только генерации одной новой доли.

- **Гибкость развертывания:** благодаря архитектуре Cosmos SDK, решение может быть развернуто как в публичных сетях (с соответствующими затратами газа), так и в частных корпоративных блокчейнах с настраиваемыми параметрами производительности.

6.2. Анализ безопасности

Теоретический анализ безопасности (раздел 5) подтверждает, что система сохраняет конфиденциальность при компрометации до $k - 1$ guardians и остаётся функциональной при отказе до $n - k$ участников. Для конфигурации (3, 5) вероятность компрометации при независимой вероятности компрометации каждого guardian $p = 0.1$ составляет:

$$P_{comp} = \sum_{i=3}^5 \binom{5}{i} p^i (1-p)^{5-i} \approx 0.00856$$

что обеспечивает высокий уровень защиты (<1% риск компрометации).

7. Направления дальнейших исследований

Предложенная архитектура открывает следующие перспективные направления развития:

- 1) **Прикладные сценарии:** адаптация для корпоративного управления ключами, резервного копирования криптовалютных кошельков и управления доступом к конфиденциальным документам в распределённых системах.
- 2) **Гибридные модели доверия:** интеграция с Web2 OAuth-провайдерами в качестве дополнительного слоя аутентификации guardians с привязкой к системе DID [16].
- 3) **Zero-knowledge доказательства:** верификация корректности вычислений при восстановлении секрета без раскрытия промежуточных данных.
- 4) **Кросс-чейн восстановление:** распределение guardians по различным блокчейн-сетям через IBC и Interchain Accounts для повышения устойчивости.

8. Заключение

В данной работе предложена и реализована мультиагентная схема восстановления пользовательских данных в экосистеме Cosmos на основе порогового разделения секрета Шамира.

Ключевые результаты работы:

- 1) В отличие от существующих решений (Argent/Pedin восстанавливают доступ к аккаунту и активам, Lear/Kepir используют централизованный OAuth), предложенная система восстанавливает зашифрованные пользовательские данные через децентрализованную сеть guardians на блокчейне Cosmos без зависимости от Web2.
- 2) Доказана устойчивость к компрометации до $k - 1$ участников; для конфигурации (3, 5) вероятность компрометации < 1% при $p = 0.1$. Вычислительная сложность восстановления составляет $O(k^2)$, что приемлемо для $k \leq 10$.
- 3) Разработан прототип на CosmWasm с веб-интерфейсом. Открытый исходный код обеспечивает воспроизводимость и развертывание в различных конфигурациях Cosmos-сетей.

Предложенная архитектура расширяет возможности управления данными в Cosmos. Перспективными направлениями являются интеграция с IBC и Interchain Accounts для кросс-чейн сценариев, применение в корпоративных системах, и интеграция с DID для реализации Self-Sovereign Identity.

Список литературы

1. *Argent Labs*. Argent: The Most Simple and Safe Crypto Wallet. Argent Documentation. 2023. URL: <https://www.argent.xyz>
2. *Buterin V.* EIP-86: Abstraction of transaction origin and signature. Ethereum Improvement Proposals. 2017. URL: <https://eips.ethereum.org/EIPS/eip-86>
3. *Buterin V.* EIP-1014: Skinny CREATE2. Ethereum Improvement Proposals. 2018. URL: <https://eips.ethereum.org/EIPS/eip-1014>
4. *Buterin V., Dietrichs A., Garnett M., Villanueva W., Wilson S.* EIP-2938: Account Abstraction. Ethereum Improvement Proposals. 2020. URL: <https://eips.ethereum.org/EIPS/eip-2938>
5. *Buterin V., Weiss Y., Tirosh D., Nacson S., Forshtat A., Gazso K., Hess T.* ERC-4337: Account Abstraction Using Alt Mempool. Ethereum Improvement Proposals. 2021. URL: <https://eips.ethereum.org/EIPS/eip-4337>
6. *CosmWasm Team*. CosmWasm: Smart Contracts for the Cosmos Ecosystem. CosmWasm Documentation. 2021. URL: <https://cosmwasm.com>
7. *Ethereum Community*. ERC-7093: Social Recovery Interface. Ethereum Improvement Proposals. 2023. URL: <https://eips.ethereum.org/EIPS/eip-7093>
8. *CryptoRank Research Team*. Account Abstraction: Path to Mass Adoption. CryptoRank Insights. 2023. URL: <https://cryptorank.io/insights/research/account-abstraction-path-to-mass-adoption>
9. *Buterin V.* Why We Need Wide Adoption of Social Recovery Wallets. Vitalik.ca Blog. 2021. URL: <https://vitalik.ca/general/2021/01/11/recovery.html>
10. *Shamir A.* How to share a secret // Communications of the ACM. 1979. V. 22, N 11. P. 612–613.
11. *Skibinsky M., Dodis Y., Spies T., Ahmad W.* Decentralized Storage of Crypto Assets via Hierarchical Shamir's Secret Sharing. Vault12 Whitepaper. 2018. URL: <https://github.com/vault12/whitepapers>
12. *Miller A.* Simple Security with Shamir Secret Sharing. Grid+ Blog. 2017. URL: <https://blog.gridplus.io/simple-security-with-shamir-secret-sharing>
13. *Bartolucci S., Bernat P., Joseph D.* Sharvot: Secret Share-Based Voting on the Blockchain // WETSEB '18: Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain. 2018. P. 30–34.
14. *Singh H.P., Stefanidis K., Kirstein F.* A Private Key Recovery Scheme Using Partial Knowledge // 2021 11th IFIP International Conference on New Technologies, Mobility and Security (NTMS). Paris, France. 2021. P. 1–5.
15. *Pedin A.B., Siasi N., Sameni M.* Smart Contract-Based Social Recovery Wallet Management Scheme for Digital Assets // Proceedings of the 2023 ACM Southeast Conference (ACMSE '23). 2023. P. 177–181.
16. *Allen C.* The Path to Self-Sovereign Identity. Life With Alacrity Blog. 2016. URL: <http://www.lifewithalacrity.com/2016/04/the-path-to-self-sovereign-identity.html>

References

1. *Argent Labs*. Argent: The Most Simple and Safe Crypto Wallet. Argent Documentation. 2023. URL: <https://www.argent.xyz>
2. *Buterin V.* EIP-86: Abstraction of transaction origin and signature. Ethereum Improvement Proposals. 2017. URL: <https://eips.ethereum.org/EIPS/eip-86>
3. *Buterin V.* EIP-1014: Skinny CREATE2. Ethereum Improvement Proposals. 2018. URL: <https://eips.ethereum.org/EIPS/eip-1014>
4. *Buterin V., Dietrichs A., Garnett M., Villanueva W., Wilson S.* EIP-2938: Account Abstraction. Ethereum Improvement Proposals. 2020. URL: <https://eips.ethereum.org/EIPS/eip-2938>
5. *Buterin V., Weiss Y., Tirosh D., Nacson S., Forshtat A., Gazso K., Hess T.* ERC-4337: Account Abstraction Using Alt Mempool. Ethereum Improvement Proposals. 2021. URL: <https://eips.ethereum.org/EIPS/eip-4337>
6. *CosmWasm Team*. CosmWasm: Smart Contracts for the Cosmos Ecosystem. CosmWasm Documentation. 2021. URL: <https://cosmwasm.com>
7. *Ethereum Community*. ERC-7093: Social Recovery Interface. Ethereum Improvement Proposals. 2023. URL: <https://eips.ethereum.org/EIPS/eip-7093>
8. *CryptoRank Research Team*. Account Abstraction: Path to Mass Adoption. CryptoRank Insights. 2023. URL: <https://cryptorank.io/insights/research/account-abstraction-path-to-mass-adoption>
9. *Buterin V.* Why We Need Wide Adoption of Social Recovery Wallets. Vitalik.ca Blog. 2021. URL: <https://vitalik.ca/general/2021/01/11/recovery.html>
10. *Shamir A.* How to share a secret. Communications of the ACM. 1979. V. 22, N 11. P. 612–613.
11. *Skibinsky M., Dodis Y., Spies T., Ahmad W.* Decentralized Storage of Crypto Assets via Hierarchical Shamir's Secret Sharing. Vault12 Whitepaper. 2018. URL: <https://github.com/vault12/whitepapers>
12. *Miller A.* Simple Security with Shamir Secret Sharing. Grid+ Blog. 2017. URL: <https://blog.gridplus.io/simple-security-with-shamir-secret-sharing>
13. *Bartolucci S., Bernat P., Joseph D.* Sharvot: Secret Share-Based Voting on the Blockchain. WETSEB '18: Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain. 2018. P. 30–34.
14. *Singh H.P., Stefanidis K., Kirstein F.* A Private Key Recovery Scheme Using Partial Knowledge. 2021 11th IFIP International Conference on New Technologies, Mobility and Security (NTMS). Paris, France. 2021. P. 1–5.
15. *Pedin A.B., Siasi N., Sameni M.* Smart Contract-Based Social Recovery Wallet Management Scheme for Digital Assets. Proceedings of the 2023 ACM Southeast Conference (ACMSE '23). 2023. P. 177–181.
16. *Allen C.* The Path to Self-Sovereign Identity. Life With Alacrity Blog. 2016. URL: <http://www.lifewithalacrity.com/2016/04/the-path-to-self-sovereign-identity.html>

Поступила в редакцию дд.мм.гггг

Сведения об авторах статей (на момент подачи статьи)

Безопасное хранение и восстановление пользовательских данных с помощью мультиагентной схемы в Cosmos

Кудрявцев Иван Вальтерович (Московский физико-технический институт (национальный исследовательский университет), аспирант) kudriavtsev.iv@phystech.edu

Горгадзе Владимир Вячеславович (Московский физико-технический институт (национальный исследовательский университет), заведующий кафедрой, PhD, доцент) gorgadze@gmail.com

Беленов Александр Викторович (Московский физико-технический институт (национальный исследовательский университет), преподаватель) askbela@gmail.com

Ссылки на опубликованные статьи (в соответствии с ГОСТ Р 7.0.5-2008)

Кудрявцев И.В., Горгадзе В.В., Беленов А.В. Безопасное хранение и восстановление пользовательских данных с помощью мультиагентной схемы в Cosmos // Труды МФТИ. 2025. Т. 17, № 1. С. 1–14.

Kudryavtsev I.V., Gorgadze V.V., Belenov A.V. Secure Storage and Recovery of User Data Using Multi-Agent Scheme in Cosmos // Proceedings of MIPT. 2025. V. 11, N 1. P. 1–14.